

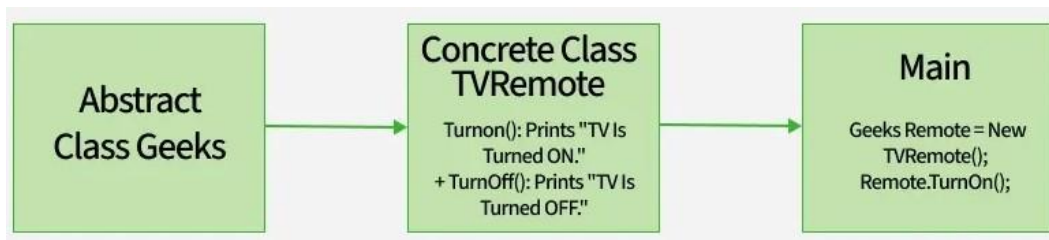
OOPS Concept

Abstraction in Java :-

- Abstraction means hiding unnecessary details and showing only essential features.
- It helps to reduce code complexity and improve maintainability.

Real-Life Example of Abstraction :-

A TV remote hides the complex working of the TV and lets us control it using simple buttons.



How to Achieve Abstraction in Java?

Java provides two ways to implement abstraction, which are listed below:

- **Abstract classes** – can have abstract and non-abstract methods.
- **Interfaces** – contain abstract methods (by default).

1. Abstract Classes (Partial Abstraction) -

- An abstract class is declared using the abstract keyword.

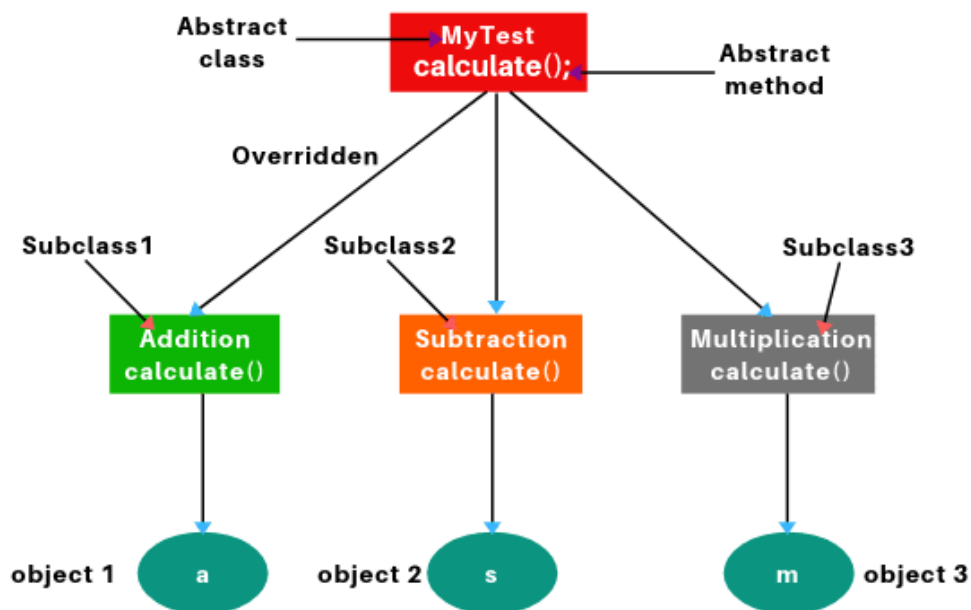


Fig: Abstract class and its subclasses

- It can have both abstract methods (without body) and non-abstract methods (with body).
- It cannot be instantiated directly.
- It is used to achieve partial abstraction.
- Subclasses must provide implementations for all abstract methods.

Example 1 –

```
package oops;

abstract class appuser {
    abstract void password();
}

package oops;

public class appimplementation extends appuser{

    @Override
    void password()
    {
        System.out.println("123456");
    }

    public static void main(String[] args) {
        appimplementation ob=new appimplementation();
        ob.password();
    }
}
```

Output –

123456

Example 2 –

```
package oops;

public interface bank {
    abstract void bank_ballance();

    default void ac_holder_name()
    {
        System.out.println("Account Holder Name is: Sagar Kate");
    }

    static void branch()
    {
        System.out.println("Branch is: Koregaon");
    }
}

package oops;

public class bank_data implements bank{

    @Override
    public void bank_ballance()
    {
        System.out.println("Bank Balance is: 234000");
    }

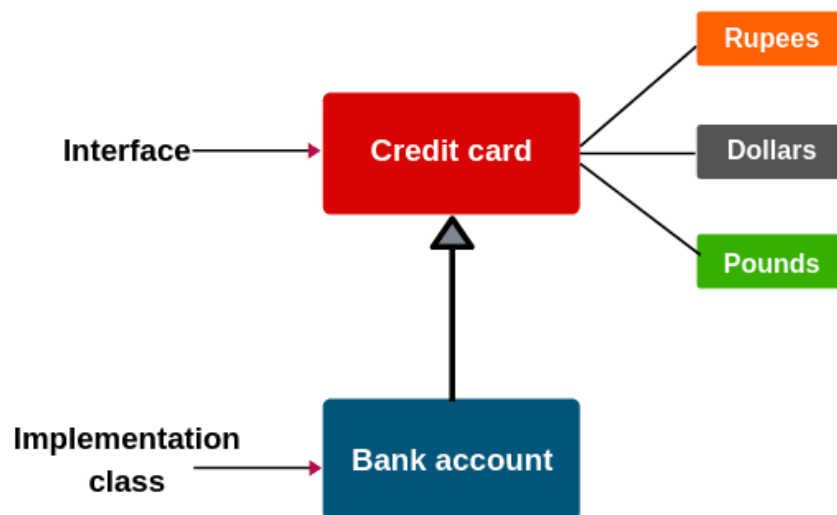
    public static void main(String[] args) {
        bank_data ob=new bank_data();
        ob.bank_ballance();
        ob.ac_holder_name();
        bank.branch();
    }
}
```

Output –

```
Bank Balance is: 234000
Account Holder Name is: Sagar Kate
Branch is: Koregaon
```

2. Interface (100% Abstraction) –

- An interface is declared using the interface keyword.
- It contains only abstract methods (by default) and constants.
- From Java 8, interfaces can also have default and static methods.
- A class implements an interface using the implements keyword.
- Interfaces provide 100% abstraction and support multiple inheritance in Java.



1. Class to Class (extends):-

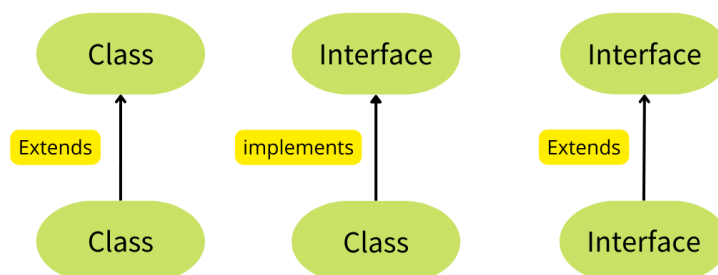
- Achieved using the extends keyword.
- A subclass inherits methods and variables from a superclass.

2. Class to Interface (implements):-

- A class implements an interface using the implements keyword.
- The class must provide implementation for all abstract methods of the interface.

3. Interface to Interface (Interface):-

- An interface can inherit another interface using the extends keyword.
- Supports multiple inheritance of interfaces.



Example – Multiple Inheritance using Interface.

```
package oops;
interface a
{
    void show();
}
interface b
{
    void disply();
}
interface c
{
    void info();
}

public class multiple_inheritance implements a,b,c{

    @Override
    public void show()
    {
        System.out.println("This is Interface A");
    }

    @Override
    public void disply()
    {
        System.out.println("This is Interface B");
    }

    @Override
    public void info()
    {
        System.out.println("This is Interface C");
    }

    public static void main(String[] args) {
        multiple_inheritance ob=new multiple_inheritance();
        ob.show();
        ob.disply();
        ob.info();
    }
}
```

Output –

```
This is Interface A
This is Interface B
This is Interface C
```

Advantages of Abstraction –

- **Hides Complexity:** Users see only essential features, not internal details.
- **Improves Security:** Implementation details are hidden, reducing misuse.
- **Enhances Code Reusability:** Abstract classes and interfaces allow reusable code.
- **Easier Maintenance:** Changes in implementation don't affect users.
- **Focus on What, Not How:** Emphasizes what an object does rather than how it does it.
- **Supports Loose Coupling:** Reduces dependency between objects and implementation.

Disadvantages of Abstraction –

- **Cannot Instantiate Abstract Classes:** Abstract classes cannot be used to create objects directly.
- **More Classes and Complexity:** Requires creating multiple classes and interfaces, which can increase design complexity.
- **Performance Overhead:** Using abstraction (especially via interfaces) can slightly reduce performance due to method calls at runtime.
- **Requires Proper Design:** Poorly designed abstraction can lead to confusing code and maintenance issues.
- **Incomplete Implementation:** Abstract classes and interfaces don't provide full implementation, so subclasses must implement abstract methods.